ASSIGNMENT-7.5

NAME: K. Ram charan HT NO: 2303A51196

BATCH:04 DATE:06-02-2026

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Lab Objectives:

• To identify and correct syntax, logic, and runtime errors inPython programs using AI tools.

• To understand common programming bugs and AI-assisted debugging suggestions.

• To evaluate how AI explains, detects, and fixes different types of coding errors.

• To build confidence in using AI to perform structured debugging practices.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

• Use AI tools to detect and correct syntax, logic, and runtime errors.

• Interpret AI-suggested bug fixes and explanations.

• Apply systematic debugging strategies supported by AI-generated insights.

Refactor buggy code using responsible and reliable programming Patterns

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

```
# Bug: Mutable default argument
def add_item(item, items=[]):
items.append(item)
return items
```

print(add_item(1))

print(add_item(2))

Expected Output: Corrected function avoids shared list bug

```
[2]
✓ 0s
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

print(add_item(1))
print(add_item(2))
print(add_item(3, ['a', 'b']))
```

```
[1]
[2]
['a', 'b', 3]
```

The `add_item` function was corrected to address the mutable default argument bug. The `items` parameter now defaults to `None`, and an empty list is initialized inside the function only if `items` is `None`. This ensures that each call to `add_item` gets a fresh list if one isn't provided, preventing unintended side effects from previous calls.

Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails.

Use AI to correct with tolerance.

 # Bug: Floating point precision issue

def check_sum():

return (0.1 + 0.2) == 0.3

print(check_sum())

Expected Output: Corrected function

```
[4]
✓ 0s
import math

def check_sum_corrected():
    return math.isclose(0.1 + 0.2, 0.3)

print(check_sum_corrected())
```

```
True
```

The `check_sum` function was corrected to address the floating-point precision issue. Due to how computers store floating-point numbers, `0.1 + 0.2` often results in a value very close to, but not exactly equal to, `0.3`.

To handle this, the `math.isclose()` function is used, which compares two floating-point numbers within a specified or default relative/absolute tolerance, returning `True` if they are considered equal within that tolerance. This ensures reliable comparisons for floating-point values.

The code has been corrected to use consistent 4-space indentation. This resolves the `IndentationError` that would occur due to mixed spaces and tabs.

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to

missing base case. Use AI to fix.

 # Bug: No base case

def countdown(n):

print(n)

return countdown(n-1)

countdown(5)

Expected Output : Correct recursion with stopping condition.

[5]
0s

```python
def countdown(n):
    if n <= 0:
        print("Done!")
        return
    print(n)
    countdown(n - 1)

countdown(5)
```

```
5
4
3
2
1
Done!
```

The `countdown` function was corrected by adding a base case: `if n <= 0: print("Done!"); return`. This ensures that the recursion stops when `n` reaches 0 or less, preventing an infinite loop. Without this base case, the function would continue to call itself indefinitely, eventually leading to a `RecursionError`.

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes

error. Use AI to fix it.

 # Bug: Accessing non-existing key

def get_value():

data = {"a": 1, "b": 2}

return data["c"]

print(get_value())

Expected Output: Corrected with .get() or error handling.

[6]
0s

```python
def get_value_corrected():
    data = {"a": 1, "b": 2}
    # Using .get() with a default value
    return data.get("c", "Key 'c' not found")

print(get_value_corrected())
```

```
Key 'c' not found
```

The `get_value` function was corrected to address the `KeyError` that occurs when trying to access a non-existent dictionary key. Instead of direct access (`data["c"]`), the `dict.get()` method is used.

`dict.get(key, default_value)` attempts to retrieve the value associated with the `key`. If the `key` is found, its corresponding value is returned. If the `key` is not found, `default_value` is returned instead of raising a `KeyError`. In this correction, if 'c' is not found, the string "Key 'c' not found" is returned.

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect

and fix it.

 # Bug: Infinite loop

def loop_example():

i = 0

while i < 5:

print(i)

Expected Output: Corrected loop increments i

[7]
0s

```python
def loop_example_corrected():
    i = 0
    while i < 5:
        print(i)
        i += 1 # Increment i to avoid infinite loop

loop_example_corrected()
```

```
0
1
2
3
4
```

The `loop_example` function was corrected to address the infinite loop bug. The original code was missing the `i += 1` statement inside the `while` loop, which meant the value of `i` never changed. As a result, the condition `i < 5` would always be true, leading to an endless loop.

By adding `i += 1`, the variable `i` is now incremented in each iteration, eventually making the condition `i < 5` false and allowing the loop to terminate correctly.

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to

fix it.

 # Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

The original code `a, b = (1, 2, 3)` causes a `ValueError: too many values to unpack (expected 2)`. This happens because you are trying to assign three values from the tuple to only two variables.

Here are a couple of ways to correct this, depending on your intent:

[8]
0s

```python
# Correction Option 1: Match the number of variables to the number of elements needed.
# If you only need the first two values:
a, b = (1, 2)
print(f"a: {a}, b: {b}")

# Or, if you want all values and assign them to separate variables:
a, b, c = (1, 2, 3)
print(f"a: {a}, b: {b}, c: {c}")
```

```
a: 1, b: 2
a: 1, b: 2, c: 3
```

[9]
0s

```python
# Correction Option 2: Use a placeholder `_` for values you don't need.
# If you only care about the first two values and want to discard the third:
a, b, _ = (1, 2, 3)
print(f"a: {a}, b: {b}")

# Or, to capture multiple remaining items into a list using the * operator:
a, *rest = (1, 2, 3, 4, 5)
print(f"a: {a}, rest: {rest}")
```

```
a: 1, b: 2
a: 1, rest: [2, 3, 4, 5]
```

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks

execution. Use AI to fix it.

 # Bug: Mixed indentation

def func():

x = 5

y = 10

return x+y

*Expected Output : Consistent indentation applied***.**

Gemini

[1]
0s

```python
def func():
    x = 5
    y = 10
    return x+y

# You can call the function to test it
result = func()
print(result)
```

```
15
```

The code has been corrected to use consistent 4-space indentation. This resolves the `IndentationError` that would occur due to mixed spaces and tabs.

Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

 # Bug: Wrong import

import maths

print(maths.sqrt(16))

Expected Output: Corrected to import math

[10]
✓ 0s

```
import math

print(math.sqrt(16))
```

```
4.0
```

The original code had an `ImportError` because it tried to import a non-existent module named `maths`. The correct module for mathematical functions in Python is `math`.

This correction changes `import maths` to `import math`, allowing the program to correctly access the `sqrt()` function (and other mathematical functions) from the standard Python `math` library.